

Universidad Tecnológica del Perú
Ingeniería de Software
Escuela de Ingeniería



"tipos de pruebas con TDD"

tarea que como parte del curso Integrador I

Presentan los alumnos:

Aulla Gonzales, Jacson Michael

Millan Pariona, Mauricio Leandro

Docente:

Mg. Anita Condo López

Lima - Perú

2025

Pruebas Unitarias

Son las más pequeñas y básicas dentro del desarrollo guiado por pruebas. Su objetivo principal es verificar que una función, clase o método cumpla exactamente con lo que se espera de ella de manera aislada, sin depender de otros componentes. Al enfocarse en piezas individuales del código, permiten detectar errores de forma temprana y facilitan la refactorización, ya que cualquier cambio se puede validar rápidamente ejecutando las pruebas. Normalmente se utilizan frameworks de testing que automatizan este proceso.

Pruebas de Integración

Validan la interacción entre distintos módulos, componentes o servicios del sistema. Aunque cada módulo haya sido probado individualmente mediante pruebas unitarias, es posible que al comunicarse entre ellos surjan errores de compatibilidad, formatos de datos incorrectos o fallos de lógica. Las pruebas de integración buscan garantizar que las partes funcionen de manera conjunta y coordinada, asegurando que la aplicación avance de forma estable al combinar diferentes capas como la lógica de negocio, la base de datos y servicios externos.

Pruebas de Aceptación

Comprueban que el sistema cumpla los requisitos definidos por el cliente o usuario final. Estas pruebas se centran en el comportamiento global de la aplicación desde la perspectiva de negocio, más que en los detalles técnicos. A menudo se basan en escenarios reales descritos en historias de usuario, por ejemplo: “Un usuario debe poder registrarse, recibir un correo de confirmación e iniciar sesión correctamente”. Suelen automatizarse con herramientas de BDD (Behavior Driven Development) como Cucumber o Behave, y permiten validar que el

software realmente cumple con las expectativas del producto.

Pruebas de Regresión

Detectan si un cambio nuevo rompe funcionalidades que antes funcionaban correctamente. Son fundamentales en proyectos en evolución, ya que cada actualización de código puede introducir fallos inesperados en áreas que parecían estables. Estas pruebas se apoyan en la suite de pruebas existentes (unitarias, de integración y aceptación), que actúan como una red de seguridad. En el enfoque TDD, las pruebas de regresión aparecen de forma natural, porque cada nueva prueba que se agrega se mantiene en el sistema y se ejecuta continuamente para asegurar que los cambios no dañen el comportamiento previo.

Pruebas End-to-End (E2E)

Simulan el recorrido completo de un usuario a través del sistema, desde la interfaz hasta la base de datos y los servicios externos. A diferencia de las pruebas unitarias o de integración, aquí se busca validar la experiencia real, comprobando que todos los componentes interactúan correctamente en conjunto. Son especialmente útiles para garantizar que los procesos críticos de negocio, como un flujo de compra o de inicio de sesión, funcionen sin errores en un entorno lo más parecido posible al de producción. Aunque suelen ser más lentas de ejecutar, aportan una gran seguridad al probar escenarios reales de extremo a extremo.

Pruebas con Mocks, Stubs y Fakes

Se basan en el uso de objetos simulados que reemplazan dependencias externas del sistema, como servicios de terceros, APIs o bases de datos. Su objetivo es aislar la lógica que

se desea probar y evitar factores externos que puedan ralentizar o complicar la ejecución de las pruebas.

- **Mocks:** Simulan objetos y permiten verificar si ciertos métodos fueron llamados con los parámetros correctos.
- **Stubs:** Devuelven respuestas predefinidas a llamadas específicas, sin lógica interna compleja.
- **Fakes:** Implementaciones ligeras y simplificadas que sustituyen a componentes reales, como una base de datos en memoria.